PYTHON

PRE-ML REFERENCE

COMPREHENSIVE ANNOTATED CHEAT SHEET

Zero baseline → ML-ready · Core Python · NumPy · Pandas · Matplotlib · scikit-learn

TABLE OF CONTENTS

SECTION 1: CORE PYTHON

■ 1.1 Variables & Data Types

Python is dynamically typed — no type declarations required. Every value is an object.

```
# Assignment
x = 42            # int
pi = 3.14159      # float
name = 'Alice'    # str
flag = True       # bool  (True / False — capital)
nothing = None    # NoneType — represents absence of value

# Type inspection
type(x)           # <class 'int'>
isinstance(x, int)  # True

# Explicit casting
float(42)         # 42.0
int(3.9)          # 3  (truncates, does NOT round)
str(100)          # '100'
bool(0)           # False  (0, '', [], None → False; everything else → True)
```

■ *ML RELEVANCE: All ML features are floats or ints. Knowing casting rules prevents silent dtype bugs in NumPy arrays.*

Type	Example	Notes
`int`	42, -7, 0	Arbitrary precision
`float`	3.14, 1e-5	64-bit IEEE 754
`str`	'hello'	Immutable sequence of chars
`bool`	True / False	Subclass of int (True==1, False==0)
`NoneType`	None	Singleton; signals missing value
`complex`	2+3j	Real+imaginary; rare in ML

■ 1.2 Strings

Strings are immutable sequences. Indexing, slicing, and a rich method library cover all text processing.

```
s = 'Hello, World!'

# Indexing (0-based, negatives count from end)
s[0]       # 'H'
s[-1]      # '!'

# Slicing  [start:stop:step]  (stop is exclusive)
s[0:5]     # 'Hello'
s[7:]      # 'World!'
s[::-1]    # '!dlroW ,olleH'  (reversed)

# f-strings (Python 3.6+) — preferred for formatting
name = 'Alice'; score = 0.9743
f'User {name} scored {score:.2%}'   # 'User Alice scored 97.43%'

# Common methods
s.lower()          # 'hello, world!'
s.upper()          # 'HELLO, WORLD!'
s.strip()          # removes leading/trailing whitespace
s.split(', ')      # ['Hello', 'World!']
', '.join(['a','b','c'])   # 'a, b, c'
s.replace('World', 'ML')   # 'Hello, ML!'
s.startswith('Hello')      # True
'World' in s               # True
len(s)                     # 13
```

■ *ML RELEVANCE: f-strings are used extensively for logging training metrics. split/join are critical for CSV parsing without Pandas.*

■ 1.3 Lists

Lists are mutable, ordered, heterogeneous sequences. Foundation of all iteration in Python.

```
lst = [10, 20, 30, 40, 50]

# Access & slice (same rules as strings)
lst[1]          # 20
lst[1:4]        # [20, 30, 40]
lst[-2:]        # [40, 50]

# Mutation
lst.append(60)          # [10, 20, 30, 40, 50, 60]
lst.insert(0, 0)        # [0, 10, 20, ...]
lst.remove(10)          # removes first occurrence of 10
lst.pop()               # removes & returns last element
lst.pop(0)              # removes & returns index 0
lst.extend([70, 80])    # in-place concatenation
lst.sort()              # in-place sort (ascending)
lst.sort(reverse=True)  # descending
sorted(lst)             # returns new sorted list
lst.reverse()           # in-place reverse

# Info
len(lst)          # length
lst.count(20)     # occurrences of 20
lst.index(30)     # index of first 30
30 in lst         # True

# Concatenation & repetition
[1,2] + [3,4]    # [1, 2, 3, 4]
[0] * 5          # [0, 0, 0, 0, 0]  ← useful for initializing

# Unpacking
a, b, c = [1, 2, 3]
first, *rest = [1, 2, 3, 4]   # first=1, rest=[2,3,4]
```

■ *ML RELEVANCE: Lists store batch samples, labels, and intermediate results before converting to NumPy arrays. *rest unpacking is used in data pipeline splits.*

■ 1.4 Tuples

Tuples are immutable lists. Used for fixed-size records, function return values, and dictionary keys.

```
t = (1, 2, 3)
t[0]          # 1
t[-1]         # 3
len(t)        # 3

# Unpacking
x, y, z = t

# One-element tuple requires trailing comma
single = (42,)   # NOT (42) — that's just 42

# Tuples as dict keys (lists cannot be dict keys)
coords = {(0,0): 'origin', (1,0): 'x-axis'}

# Named tuples (structured records)
from collections import namedtuple
Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 4)
p.x   # 3
```

■ *ML RELEVANCE: Model outputs (loss, accuracy) are often returned as tuples. Array shapes in NumPy/PyTorch are tuples.*

■ 1.5 Dictionaries

Dicts are mutable key→value mappings with O(1) average-case lookup. Keys must be hashable (immutable).

```
d = {'name': 'Alice', 'age': 30, 'score': 0.95}

# Access
d['name']              # 'Alice'  (KeyError if missing)
d.get('name')          # 'Alice'  (None if missing — safe)
d.get('missing', -1)   # -1       (custom default)

# Mutation
d['email'] = 'a@b.com'  # add/update
del d['age']             # delete key
d.pop('score', None)     # remove & return (safe if missing)

# Iteration
d.keys()                 # dict_keys(['name', ...])
d.values()               # dict_values([...])
d.items()                # dict_items([('name','Alice'), ...])

for key, val in d.items():
    print(f'{key}: {val}')

# Merging (Python 3.9+)
merged = d1 | d2         # new merged dict
d1 |= d2                 # in-place merge

# Checking membership
'name' in d              # True (checks keys)
```

■ *ML RELEVANCE: Hyperparameter configs, metric logs (loss per epoch), class label mappings, and vocabulary indices are all stored as dicts.*

■ 1.6 Sets

Unordered collections of unique, hashable elements. O(1) membership testing.

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}

s1.add(5)          # {1,2,3,4,5}
s1.remove(5)       # KeyError if missing
s1.discard(99)     # no error if missing

# Set operations
s1 | s2            # union         {1,2,3,4,5,6}
s1 & s2            # intersection  {3,4}
s1 - s2            # difference    {1,2}
s1 ^ s2            # symmetric diff {1,2,5,6}

3 in s1            # True — O(1)
len(s1)            # 4

# Convert list to set to remove duplicates
unique = list(set([1,1,2,2,3]))   # [1,2,3]  (order not guaranteed)
```

■ *ML RELEVANCE: Sets are used for vocabulary deduplication, computing unique class labels, and checking dataset overlap between train/test splits.*

■ 1.7 Control Flow

```
# if / elif / else
x = 85
if x >= 90:
    grade = 'A'
elif x >= 80:
    grade = 'B'
elif x >= 70:
    grade = 'C'
else:
    grade = 'F'

# Ternary (inline if)
label = 'pass' if x >= 70 else 'fail'

# Walrus operator := (Python 3.8+) — assign and test
if (n := len(data)) > 100:
    print(f'Large dataset: {n} samples')

# Truthiness rules
# Falsy:  None, 0, 0.0, '', [], {}, set(), False
# Truthy: everything else
if data:   # equivalent to len(data) > 0
    process(data)
```

■ *ML RELEVANCE: Conditional logic drives early stopping, threshold-based classification decisions, and learning rate scheduling checks.*

■ 1.8 Loops

```
# for loop — iterates any iterable
for i in range(5):          # 0,1,2,3,4
    print(i)

for i in range(2, 10, 2):   # 2,4,6,8  (start,stop,step)
    print(i)

# enumerate — index + value simultaneously
fruits = ['apple', 'banana', 'cherry']
for idx, val in enumerate(fruits):
    print(idx, val)         # 0 apple, 1 banana ...

# zip — iterate multiple sequences in parallel
for x, y in zip([1,2,3], [4,5,6]):
    print(x, y)             # 1 4, 2 5, 3 6

# while loop
epoch = 0
while epoch < 100:
    train()
    epoch += 1

# Loop control
for i in range(10):
    if i == 3: continue     # skip rest of this iteration
    if i == 7: break        # exit loop entirely
    print(i)

# for...else: else runs if loop completes WITHOUT break
for item in data:
    if item < 0:
        print('negative found')
        break
else:
    print('all values non-negative')
```

■ *ML RELEVANCE: Training loops iterate epochs and batches. enumerate is used to track batch indices. zip pairs inputs with labels.*

■ 1.9 Functions

```
# Basic definition
def add(a, b):
    return a + b

# Default arguments
def greet(name, greeting='Hello'):
    return f'{greeting}, {name}!'

greet('Alice')            # 'Hello, Alice!'
greet('Alice', 'Hi')      # 'Hi, Alice!'

# Keyword arguments
def model(lr=0.01, epochs=10, batch_size=32):
    pass

model(epochs=50, lr=0.001)   # order doesn't matter

# *args — variable positional arguments → tuple
def total(*args):
    return sum(args)

total(1, 2, 3, 4)   # 10

# **kwargs — variable keyword arguments → dict
def config(**kwargs):
    for k, v in kwargs.items():
        print(f'{k} = {v}')

config(lr=0.001, dropout=0.5)   # lr = 0.001, dropout = 0.5

# Multiple return values (returns tuple)
def stats(data):
    return min(data), max(data), sum(data)/len(data)

lo, hi, mean = stats([1,2,3,4,5])

# Type hints (recommended for ML code)
def normalize(x: float, mean: float, std: float) -> float:
    return (x - mean) / std

# Docstrings
def predict(X):
    '''Run forward pass on input X. Returns predicted labels.'''
    pass
```

■ *ML RELEVANCE: ML functions use *args/**kwargs for flexible hyperparameter APIs. Type hints document tensor shapes and dtypes. Multiple returns handle (loss, accuracy) pairs.*

■ 1.10 Lambda & Higher-Order Functions

```
# Lambda: anonymous single-expression function
square = lambda x: x ** 2
square(5)   # 25

# Common use: sort by custom key
data = [('Alice', 0.95), ('Bob', 0.72), ('Carol', 0.88)]
data.sort(key=lambda x: x[1], reverse=True)  # sort by score desc

# map — apply function to every element → iterator
squares = list(map(lambda x: x**2, [1,2,3,4,5]))
# [1, 4, 9, 16, 25]

# filter — keep elements where function returns True → iterator
evens = list(filter(lambda x: x % 2 == 0, range(10)))
# [0, 2, 4, 6, 8]

# reduce — fold sequence to single value
from functools import reduce
product = reduce(lambda a, b: a * b, [1,2,3,4,5])  # 120

# Functions as first-class objects
def apply(func, data):
    return [func(x) for x in data]

apply(lambda x: x**2, [1,2,3])   # [1, 4, 9]
```

■ *ML RELEVANCE: Higher-order functions power data transformation pipelines. Lambdas are used in Pandas .apply() calls and custom loss functions.*

■ 1.11 Comprehensions

```
# List comprehension: [expr for item in iterable if condition]
squares = [x**2 for x in range(10)]
evens   = [x for x in range(20) if x % 2 == 0]
matrix  = [[i*j for j in range(4)] for i in range(4)]   # 2D

# Dict comprehension
word_len = {word: len(word) for word in ['apple','cat','banana']}
# {'apple': 5, 'cat': 3, 'banana': 6}

# Set comprehension
unique_lengths = {len(w) for w in ['apple','cat','banana','fig']}
# {3, 5, 6}

# Generator expression (lazy — does NOT build list in memory)
gen = (x**2 for x in range(1_000_000))   # no RAM spike
next(gen)   # 0
sum(x**2 for x in range(1000))   # 332833500  (memory-efficient)
```

■ *ML RELEVANCE: List comprehensions replace explicit loops for feature extraction, one-hot encoding, and batch preprocessing. Generator expressions are critical when datasets exceed RAM.*

■ 1.12 Error Handling

```
# try / except / else / finally
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f'Error: {e}')          # catches specific error
except (TypeError, ValueError):
    print('Type or value error')  # multiple exception types
except Exception as e:
    print(f'Unexpected: {e}')     # catch-all (use sparingly)
else:
    print('Success:', result)      # runs if NO exception
finally:
    print('Always runs')           # cleanup code

# Raising exceptions
def load(path):
    if not path.endswith('.csv'):
        raise ValueError(f'Expected .csv, got: {path}')

# Common ML-relevant exceptions
# FileNotFoundError   — data file missing
# KeyError            — column not found in DataFrame
# IndexError          — array subscript out of range
# ValueError          — wrong shape passed to model
# TypeError           — wrong dtype (e.g., str instead of float)
# MemoryError         — dataset too large for RAM

# Context managers (with statement) — auto cleanup
with open('data.txt', 'r') as f:
    content = f.read()   # file auto-closes even if error
```

■ *ML RELEVANCE: Robust pipelines wrap data loading and model training in try/except to log errors without crashing long training runs.*

■ 1.13 File I/O

```
# Read entire file
with open('data.txt', 'r') as f:
    text = f.read()

# Read line by line (memory-efficient for large files)
with open('data.txt', 'r') as f:
    for line in f:
        process(line.strip())

# Write
with open('output.txt', 'w') as f:
    f.write('Hello\n')

# Append
with open('log.txt', 'a') as f:
    f.write(f'epoch 1: loss=0.42\n')

# CSV — standard library
import csv
with open('data.csv', 'r') as f:
    reader = csv.DictReader(f)
    rows = list(reader)   # list of dicts per row

# JSON
import json
with open('config.json', 'r') as f:
    cfg = json.load(f)         # parse JSON → dict

with open('config.json', 'w') as f:
    json.dump(cfg, f, indent=2)  # dict → JSON file

# os.path utilities
import os
os.path.exists('file.txt')   # True/False
os.path.join('data', 'train', 'X.npy')  # cross-platform path
os.listdir('data/')          # list directory contents
os.makedirs('data/cache', exist_ok=True)  # create dirs
```

■ *ML RELEVANCE: ML workflows read/write: dataset CSVs, JSON hyperparameter configs, model weights (.pt/.pkl), and training logs. Path handling via os.path prevents Windows/Linux path bugs.*

■ 1.14 Classes & Object-Oriented Programming

```
class Animal:
    # Class variable (shared across all instances)
    kingdom = 'Animalia'

    # __init__ — constructor, called on instantiation
    def __init__(self, name: str, weight: float):
        self.name = name      # instance variable
        self.weight = weight

    # Instance method
    def describe(self) -> str:
        return f'{self.name} ({self.weight} kg)'

    # Static method — no access to self or cls
    @staticmethod
    def breathes() -> bool:
        return True

    # Class method — receives class, not instance
    @classmethod
    def from_dict(cls, d: dict):
        return cls(d['name'], d['weight'])

    # Special methods (dunder)
    def __repr__(self) -> str:
        return f'Animal({self.name!r}, {self.weight})'

    def __len__(self):
        return int(self.weight)

# Inheritance
class Dog(Animal):
    def __init__(self, name, weight, breed):
        super().__init__(name, weight)  # call parent __init__
        self.breed = breed

    def describe(self) -> str:          # override parent method
        return f'{super().describe()} — {self.breed}'

# Usage
d = Dog('Rex', 30, 'Labrador')
d.describe()    # 'Rex (30 kg) — Labrador'
isinstance(d, Animal)  # True  (Dog IS-A Animal)
```

■ *ML RELEVANCE: PyTorch models subclass nn.Module with the same inheritance pattern. Datasets subclass torch.utils.data.Dataset. Understanding OOP is mandatory for framework usage.*

■ 1.15 Modules & Imports

```
# Import entire module
import math
math.sqrt(16)     # 4.0
math.pi           # 3.141592...

# Import specific names
from math import sqrt, pi, ceil, floor, log, exp
sqrt(25)          # 5.0

# Alias (standard ML conventions — ALWAYS use these aliases)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics

# Import from submodule
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Relative import (inside packages)
from . import utils          # sibling module
from ..data import loader    # parent package

# Inspect module contents
dir(math)         # list all attributes/functions
help(math.sqrt)   # docstring
```

■ *ML RELEVANCE: np, pd, plt are universal aliases in the ML community. Code without these aliases will be unreadable to collaborators.*

■ 1.16 Iterators & Generators

```
# An iterator implements __iter__ and __next__
it = iter([1, 2, 3])
next(it)   # 1
next(it)   # 2

# Generator function — uses yield instead of return
def batch_generator(data, batch_size=32):
    for i in range(0, len(data), batch_size):
        yield data[i : i + batch_size]   # suspends here

for batch in batch_generator(X_train, 32):
    train_step(batch)   # processes 32 samples at a time

# Generator with send() — coroutine pattern
def running_avg():
    total, count = 0, 0
    while True:
        val = yield total / count if count else 0
        total += val; count += 1

# itertools — powerful iterator building blocks
import itertools
list(itertools.chain([1,2], [3,4], [5]))   # [1,2,3,4,5]
list(itertools.islice(range(1000), 5))     # [0,1,2,3,4]
list(itertools.product('AB', [1,2]))        # combinations
```

■ *ML RELEVANCE: Batch generators are the foundation of custom PyTorch DataLoaders. Using generators instead of lists prevents out-of-memory errors on large datasets.*

■ 1.17 Built-in Functions Reference

Function	Description	ML Use Case
`len(x)`	Length of sequence/collection	Check dataset size
`range(n)`	Integer sequence [0,n)	Epoch/batch loops
`enumerate(it)`	Yields (index, value) pairs	Track batch index
`zip(*iters)`	Pair-wise iteration	Pair inputs & labels
`map(f, it)`	Apply f to each element	Feature transforms
`filter(f, it)`	Keep elements where f is True	Sample filtering
`sorted(it)`	Return sorted list	Rank predictions
`min/max(it)`	Minimum/maximum	Sanity checks
`sum(it)`	Sum of elements	Loss accumulation
`abs(x)`	Absolute value	Error metrics
`round(x, n)`	Round to n decimals	Metric display
`any(it)`	True if any element truthy	Convergence checks
`all(it)`	True if all elements truthy	Validation checks
`isinstance(x,T)`	Type check	Input validation
`type(x)`	Return type object	Debugging dtypes
`print(*args)`	Output to stdout	Logging metrics
`open(path,mode)`	File handle	Data I/O
`dict/list/set/tuple`	Type constructors	Type conversions
`zip(*lst)`	Unzip (transpose) list	Batch unpacking
`vars(obj)`	Return __dict__	Inspect model params

SECTION 2: NUMPY

NumPy provides the N-dimensional array (ndarray) — the universal data container for numerical ML. All ML frameworks (PyTorch, TensorFlow, scikit-learn) interoperate with NumPy arrays.

```
import numpy as np
```

■ 2.1 Array Creation

```
# From Python sequences
a = np.array([1, 2, 3, 4])              # 1D, dtype inferred
b = np.array([[1,2],[3,4]], dtype=float) # 2D, explicit dtype

# Zeros, ones, constants
np.zeros((3, 4))           # 3x4 matrix of 0.0
np.ones((2, 3))            # 2x3 matrix of 1.0
np.full((3, 3), 7)         # 3x3 matrix of 7
np.eye(4)                  # 4x4 identity matrix
np.empty((100, 50))        # uninitialized (fast, but random values)

# Sequences
np.arange(0, 10, 2)        # [0 2 4 6 8]  (start, stop, step)
np.linspace(0, 1, 5)       # [0.   0.25 0.5  0.75 1.]  (N evenly spaced)
np.logspace(0, 3, 4)       # [1.  10. 100. 1000.]  (log scale)

# From existing
np.copy(a)                  # explicit copy (modifying won't affect a)
np.zeros_like(a)            # zeros with same shape & dtype as a
np.ones_like(a)
```

■ *ML RELEVANCE: np.zeros/ones initialize weight matrices and bias vectors. np.linspace generates learning rate schedules and decision boundaries.*

■ 2.2 Array Attributes

```
a = np.array([[1,2,3],[4,5,6]], dtype=np.float32)

a.shape      # (2, 3)  — (rows, cols)
a.ndim       # 2       — number of dimensions
a.size       # 6       — total number of elements
a.dtype      # float32 — element data type
a.nbytes     # 24      — total bytes in memory
a.itemsize   # 4       — bytes per element

# Common dtypes
# np.float32  — standard for neural nets (GPU-friendly, half RAM of float64)
# np.float64  — Python default float (double precision)
# np.int32    — integer labels
# np.int64    — large integer indices
# np.bool_    — masks
# np.uint8    — image pixel values (0-255)

# Change dtype
a.astype(np.float64)    # returns new array — does NOT modify in-place
```

■ *ML RELEVANCE: Always check .shape and .dtype before feeding arrays to models. Shape mismatches are the #1 source of ML runtime errors. float32 saves ~50% GPU memory vs float64.*

■ 2.3 Indexing & Slicing

```
a = np.array([[10,20,30],
              [40,50,60],
              [70,80,90]])

# Integer indexing
a[0]         # [10 20 30]  — first row (1D)
a[1, 2]      # 60          — row 1, col 2
a[-1]        # [70 80 90]  — last row

# Slicing  [row_start:row_stop, col_start:col_stop]
a[:, 0]      # [10 40 70]  — all rows, col 0
a[0:2, 1:]   # [[20 30],[50 60]]
a[::2]       # [[10 20 30],[70 80 90]]  — every 2nd row

# Boolean indexing (masking)
mask = a > 40                  # bool array, same shape
a[mask]                        # [50 60 70 80 90]  — 1D
a[a % 2 == 0]                  # select even elements

# Fancy indexing (integer array indexing)
rows = np.array([0, 2])
a[rows]                        # rows 0 and 2
a[[0,1], [2,0]]                # a[0,2]=30 and a[1,0]=40

# CAUTION: slices are VIEWS (share memory), not copies
view = a[0]
view[0] = 999   # modifies a[0,0] too!
copy = a[0].copy()   # explicit copy to avoid this
```

■ *ML RELEVANCE: Boolean masking filters samples by condition (e.g., select all samples of class 0). Fancy indexing is used to shuffle datasets by permuted index arrays.*

■ 2.4 Reshaping & Stacking

```
a = np.arange(12)

# Reshape — total elements must match
a.reshape(3, 4)          # (3,4) — row-major (C order)
a.reshape(3, 4, order='F')  # Fortran (column-major) order
a.reshape(-1)            # flatten to 1D  (-1 = infer)
a.reshape(2, -1)         # (2,6) — infer second dim

# Flatten — always returns copy
a.reshape(3,4).flatten()

# Ravel — returns flattened view if possible
a.reshape(3,4).ravel()

# Transpose
b = np.array([[1,2,3],[4,5,6]])  # shape (2,3)
b.T         # shape (3,2)
b.transpose()  # same

# Add/remove dimensions
c = np.array([1,2,3])   # shape (3,)
c[np.newaxis, :]        # shape (1,3)  — add row dim
c[:, np.newaxis]        # shape (3,1)  — add col dim
np.expand_dims(c, axis=0)   # equivalent
np.squeeze(np.array([[[1,2,3]]]))  # (1,1,3) → (3,)

# Stacking
x = np.array([1,2,3]); y = np.array([4,5,6])
np.stack([x, y])         # (2,3) — new axis
np.vstack([x, y])        # (2,3) — stack vertically
np.hstack([x, y])        # (6,)  — stack horizontally
np.concatenate([x,y], axis=0)  # (6,) — general
```

■ *ML RELEVANCE: reshape(-1, 1) converts 1D label arrays to column vectors required by scikit-learn. np.newaxis adds batch dimensions. Concatenate merges train/val splits.*

■ 2.5 Mathematical Operations

```
a = np.array([1.0, 4.0, 9.0, 16.0])
b = np.array([2.0, 2.0, 3.0,  4.0])

# Element-wise arithmetic
a + b          # [ 3.  6. 12. 20.]
a - b          # [-1.  2.  6. 12.]
a * b          # [ 2.  8. 27. 64.]
a / b          # [0.5  2.  3.  4.]
a ** 2         # element-wise power
a // b         # floor division
a % b          # modulo

# Universal functions (ufuncs) — element-wise
np.sqrt(a)     # [1. 2. 3. 4.]
np.exp(a)      # e^element
np.log(a)      # natural log
np.log2(a)     # log base 2
np.log10(a)    # log base 10
np.abs(a)      # absolute value
np.sin(a); np.cos(a); np.tan(a)
np.clip(a, 0, 5)    # clamp to [0,5]
np.where(a > 4, 1, 0)  # conditional element-wise

# Dot product & matrix multiplication
np.dot(a, b)            # scalar dot product
A = np.random.randn(3, 4)
B = np.random.randn(4, 5)
np.dot(A, B)            # (3,5) matrix product
A @ B                   # identical — preferred syntax
```

■ *ML RELEVANCE: np.exp/log implement softmax and cross-entropy. @ is the matrix multiply used in every linear layer. np.clip prevents gradient explosion in manual implementations.*

■ 2.6 Broadcasting

```
# Broadcasting: NumPy automatically expands dimensions
# to make shapes compatible — WITHOUT copying data.

# Rule: align shapes from RIGHT; dims match if equal or 1.
#   (3,4) + (4,)   → (3,4) + (1,4) → (3,4)  ✓
#   (3,4) + (3,)   → FAILS  ✗
#   (3,4) + (1,4)  → (3,4)  ✓

A = np.ones((3, 4))         # shape (3,4)
v = np.array([1, 2, 3, 4])  # shape (4,)
A + v      # v broadcast across each row → (3,4)

# Subtract column mean (center each feature)
X = np.random.randn(100, 5)    # 100 samples, 5 features
X_centered = X - X.mean(axis=0)  # (100,5) - (5,) → ok

# Outer product via broadcasting
a = np.array([1,2,3])[:, np.newaxis]  # (3,1)
b = np.array([10,20,30])[np.newaxis,:]  # (1,3)
a * b   # (3,3) outer product
```

■ *ML RELEVANCE: Normalization (X - mean) / std uses broadcasting. Loss computation between (batch, classes) predictions and (batch,) labels relies on it.*

■ 2.7 Aggregation Functions

```
X = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]], dtype=float)

# Global (no axis) — single scalar
np.sum(X)        # 45.0
np.mean(X)       # 5.0
np.std(X)        # standard deviation
np.var(X)        # variance
np.min(X)        # 1.0
np.max(X)        # 9.0
np.median(X)     # 5.0

# axis=0 → collapse rows (result: 1 value per column)
np.sum(X, axis=0)    # [12. 15. 18.]
np.mean(X, axis=0)   # [ 4.  5.  6.]

# axis=1 → collapse columns (result: 1 value per row)
np.sum(X, axis=1)    # [ 6. 15. 24.]
np.max(X, axis=1)    # [ 3.  6.  9.]

# keepdims — maintain original number of dimensions
np.mean(X, axis=0, keepdims=True)   # shape (1,3), not (3,)

# Indices
np.argmin(X)         # flat index of minimum
np.argmax(X, axis=1) # index of max in each row
np.argsort(X, axis=1)  # indices that sort each row

# Cumulative
np.cumsum([1,2,3,4])    # [1 3 6 10]
np.cumprod([1,2,3,4])   # [1 2 6 24]

# Percentile
np.percentile(X, 75)    # 75th percentile
```

■ *ML RELEVANCE: axis=0 computes per-feature statistics for normalization. argmax converts probability vectors to class predictions. argsort ranks predictions by confidence.*

■ 2.8 Linear Algebra

```
import numpy.linalg as LA

A = np.array([[2.,1.],[5.,3.]])

# Determinant
LA.det(A)          # 1.0

# Inverse
LA.inv(A)          # [[3.,-1.],[-5.,2.]]

# Solve linear system Ax = b
b = np.array([1., 2.])
x = LA.solve(A, b)   # preferred over inv(A) @ b (numerically stable)

# Norms
v = np.array([3., 4.])
LA.norm(v)           # 5.0   (L2 / Euclidean)
LA.norm(v, ord=1)    # 7.0   (L1 / Manhattan)
LA.norm(v, ord=np.inf)  # 4.0  (L-inf / max norm)

# Eigenvalues & eigenvectors
vals, vecs = LA.eig(A)   # vecs: columns are eigenvectors

# SVD — Singular Value Decomposition
U, S, Vt = LA.svd(A)    # A = U @ diag(S) @ Vt

# QR decomposition
Q, R = LA.qr(A)

# Matrix rank
LA.matrix_rank(A)    # 2

# Trace
np.trace(A)          # sum of diagonal = 2+3 = 5
```

■ *ML RELEVANCE: SVD underlies PCA (dimensionality reduction). Eigendecomposition underlies covariance analysis. LA.solve trains linear regression exactly. Norms regularize model weights.*

■ 2.9 Random Module

```
rng = np.random.default_rng(seed=42)  # preferred API (reproducible)

# Distributions
rng.random((3, 4))          # uniform [0,1)
rng.integers(0, 10, (5,))   # random integers in [0,10)
rng.normal(0, 1, (100,))    # Gaussian: mean=0, std=1
rng.normal(loc=0, scale=1, size=(128, 256))  # weight init
rng.uniform(-1, 1, (50,))   # uniform [-1,1)
rng.binomial(n=1, p=0.5, size=(100,))  # Bernoulli
rng.exponential(scale=1.0, size=(100,))

# Permutation & shuffle
rng.permutation(10)          # shuffled [0..9]
rng.permutation(X)           # shuffled rows of X (returns copy)

# Shuffle in-place
arr = np.arange(10)
rng.shuffle(arr)             # shuffles arr in-place

# Dataset splitting via permutation
indices = rng.permutation(len(X))
split   = int(0.8 * len(X))
train_idx, val_idx = indices[:split], indices[split:]

# Legacy API (still common in code you'll read)
np.random.seed(42)           # global seed
np.random.randn(3, 4)        # N(0,1)
np.random.rand(3, 4)         # U[0,1)
np.random.randint(0, 10, 5)  # int [0,10)
```

■ *ML RELEVANCE: Weight initialization uses np.random.normal. Dataset shuffling before splitting prevents ordering bias. Always set seed for reproducible experiments.*

SECTION 3: PANDAS

Pandas provides two core data structures: Series (1D labeled array) and DataFrame (2D labeled table). The primary tool for data loading, cleaning, and feature engineering.

```
import pandas as pd
```

■ 3.1 Series & DataFrame Creation

```
# Series — 1D labeled array
s = pd.Series([10, 20, 30], index=['a','b','c'])
s['b']     # 20
s[1]       # 20  (positional)
s.values   # np.array([10,20,30])
s.index    # Index(['a','b','c'])

# DataFrame — 2D labeled table
df = pd.DataFrame({
    'name':  ['Alice','Bob','Carol'],
    'age':   [25, 30, 35],
    'score': [0.95, 0.72, 0.88]
})

# From list of dicts
rows = [{'x': 1,'y': 2}, {'x': 3,'y': 4}]
pd.DataFrame(rows)

# From NumPy array
arr = np.random.randn(5, 3)
pd.DataFrame(arr, columns=['A','B','C'])
```

■ *ML RELEVANCE: DataFrames load raw tabular datasets. Columns map directly to ML features. Series represent single-feature vectors or target label arrays.*

■ 3.2 Reading & Writing Data

```
# CSV
df = pd.read_csv('data.csv')                      # basic
df = pd.read_csv('data.csv', index_col=0)         # use col 0 as index
df = pd.read_csv('data.csv', header=None)         # no header row
df = pd.read_csv('data.csv', sep=';')             # delimiter
df = pd.read_csv('data.csv', nrows=1000)          # first N rows
df = pd.read_csv('data.csv', usecols=['A','B'])   # select columns
df = pd.read_csv('data.csv', dtype={'age': int})  # force dtype
df = pd.read_csv('data.csv', na_values=['-','N/A'])  # custom NaN

df.to_csv('out.csv', index=False)   # write (index=False = no row numbers)

# Excel
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
df.to_excel('out.xlsx', index=False)

# JSON
df = pd.read_json('data.json')
df.to_json('out.json', orient='records')

# Parquet (compressed, fast for large datasets)
df = pd.read_parquet('data.parquet')
df.to_parquet('out.parquet', index=False)

# Pickle (Python-only, exact preservation)
df.to_pickle('df.pkl')
df = pd.read_pickle('df.pkl')
```

■ *ML RELEVANCE: Parquet is preferred for large ML datasets (columnar, compressed, fast). CSV is universal. Pickle preserves exact dtypes including category types.*

■ 3.3 Inspection

```
df.head(5)          # first 5 rows
df.tail(5)          # last 5 rows
df.sample(10)       # 10 random rows

df.shape            # (rows, cols) tuple
df.ndim             # 2
df.size             # rows * cols
len(df)             # number of rows

df.info()           # column names, dtypes, non-null counts, memory
df.describe()       # count, mean, std, min, 25%, 50%, 75%, max
df.describe(include='all')  # include categorical cols

df.dtypes           # dtype of each column
df.columns          # column names (Index object)
df.index            # row index

df.nunique()        # number of unique values per column
df['col'].value_counts()   # frequency table for one column
df['col'].unique()         # array of unique values

df.isnull().sum()   # count NaN per column
df.isnull().any()   # True/False per column (any NaN?)
df.memory_usage(deep=True).sum()  # total bytes
```

■ *ML RELEVANCE: Always call .info() and .describe() first on any new dataset. isnull().sum() identifies missing data before preprocessing. value_counts() reveals class imbalance.*

■ 3.4 Selection & Indexing

```
# Select columns
df['name']              # Series (single column)
df[['name','score']]    # DataFrame (multiple columns)

# .loc — label-based (inclusive on both ends)
df.loc[0]               # row with index label 0
df.loc[0:2]             # rows 0,1,2  (INCLUSIVE)
df.loc[0:2, 'name']     # rows 0-2, column 'name'
df.loc[:, 'name':'score']  # all rows, cols name through score

# .iloc — position-based (exclusive on stop, like Python slices)
df.iloc[0]              # first row
df.iloc[0:3]            # rows 0,1,2
df.iloc[0:3, 0:2]       # rows 0-2, cols 0-1
df.iloc[:, -1]          # last column

# .at / .iat — single cell access (faster than loc/iloc)
df.at[0, 'name']        # label-based single cell
df.iat[0, 0]            # position-based single cell

# Setting values
df.loc[0, 'score'] = 0.99
df['grade'] = ['A','B','A']   # new column from list

# Selecting columns by dtype
df.select_dtypes(include='number')    # numeric cols only
df.select_dtypes(include='object')   # string/categorical
```

■ *ML RELEVANCE: .loc is used for label-based feature matrix slicing. .iloc is used for positional train/validation splits. Correct use prevents silent off-by-one errors.*

■ 3.5 Filtering (Boolean Indexing)

```
# Boolean mask (Series of True/False)
df[df['score'] > 0.8]                 # rows where score > 0.8
df[df['name'] == 'Alice']             # exact match
df[(df['score'] > 0.8) & (df['age'] < 30)]   # AND  (use & not 'and')
df[(df['score'] < 0.5) | (df['age'] > 40)]   # OR   (use | not 'or')
df[~(df['score'] > 0.8)]             # NOT  (use ~ not 'not')

# .isin — membership test
df[df['name'].isin(['Alice','Carol'])]

# .between — range test (inclusive)
df[df['age'].between(25, 30)]

# String pattern matching
df[df['name'].str.startswith('A')]
df[df['name'].str.contains('li', case=False)]

# .query — readable SQL-like syntax
df.query('score > 0.8 and age < 35')
thresh = 0.8
df.query('score > @thresh')   # reference local variable with @

# .where — keep value if True, else NaN (or other fill)
df['score'].where(df['score'] > 0.5, other=0.0)
```

■ *ML RELEVANCE: Boolean masking removes outliers, filters by class label, and selects train/test subsets. .query is readable for complex multi-condition filters.*

■ 3.6 Handling Missing Data

```
# Detection
df.isnull()                 # bool DataFrame — True where NaN
df.notnull()                # inverse
df.isnull().sum()           # count NaN per column
df.isnull().sum() / len(df)  # fraction NaN per column

# Dropping
df.dropna()                 # drop rows with ANY NaN
df.dropna(axis=1)           # drop columns with ANY NaN
df.dropna(how='all')        # drop rows where ALL values NaN
df.dropna(subset=['score'])  # drop rows where 'score' is NaN
df.dropna(thresh=3)         # keep rows with at least 3 non-NaN

# Filling
df.fillna(0)                # fill all NaN with 0
df['score'].fillna(df['score'].mean())    # fill with column mean
df.fillna(df.median())      # fill all numeric cols with median
df.fillna(method='ffill')    # forward fill (propagate last valid)
df.fillna(method='bfill')    # backward fill
df['cat'].fillna(df['cat'].mode()[0])    # fill categorical with mode

# Interpolation
df['value'].interpolate(method='linear')   # linear interpolation

# Replace specific values
df.replace(-1, np.nan)        # treat -1 as missing
df.replace({'?': np.nan, 'N/A': np.nan})
```

■ *ML RELEVANCE: Missing data handling is the most common data cleaning task. Mean imputation for numeric, mode for categorical. Dropping rows with NaN targets is mandatory.*

■ 3.7 GroupBy & Aggregation

```
# GroupBy — split-apply-combine pattern
df.groupby('category')['score'].mean()      # mean score per category
df.groupby('category')['score'].agg(['mean','std','count'])

# Multiple grouping keys
df.groupby(['category','subcategory'])['score'].sum()

# Named aggregations (clean column names)
df.groupby('category').agg(
    mean_score=('score', 'mean'),
    total=('score', 'count'),
    max_age=('age', 'max')
)

# Custom function
df.groupby('category')['score'].agg(lambda x: x.max()-x.min())

# Transform — return same-length result (broadcast back)
df['z_score'] = df.groupby('category')['score'].transform(
    lambda x: (x - x.mean()) / x.std()
)

# Filter — keep groups satisfying condition
df.groupby('category').filter(lambda x: len(x) >= 5)

# Iterate over groups
for name, group_df in df.groupby('category'):
    print(name, group_df.shape)
```

■ *ML RELEVANCE: GroupBy computes per-class statistics for stratified normalization and class-wise evaluation metrics. transform is used for group-wise feature engineering.*

■ 3.8 Merging & Joining

```
# merge — SQL-style join on columns
pd.merge(df1, df2, on='id')                  # inner join (default)
pd.merge(df1, df2, on='id', how='left')      # left join
pd.merge(df1, df2, on='id', how='right')     # right join
pd.merge(df1, df2, on='id', how='outer')     # full outer join

# Join on different column names
pd.merge(df1, df2, left_on='user_id', right_on='id')

# concat — stack DataFrames
pd.concat([df1, df2])              # stack rows (axis=0)
pd.concat([df1, df2], axis=1)     # stack columns (axis=1)
pd.concat([df1, df2], ignore_index=True)  # reset row index

# join — index-based join
df1.join(df2, how='left')          # join on index

# append is deprecated (use concat instead)
```

■ *ML RELEVANCE: Merging joins feature tables with label tables by ID. Concatenation assembles final feature matrices from multiple engineered feature sets.*

■ 3.9 Apply, Map & Transform

```
# apply — apply function along axis
df['score'].apply(lambda x: round(x, 2))   # element-wise on Series
df.apply(lambda col: col.max() - col.min())  # per column (axis=0 default)
df.apply(lambda row: row['score'] * 100, axis=1)  # per row

# map — element-wise on Series (simpler than apply)
df['grade'] = df['score'].map({0.95: 'A', 0.72: 'B'})
df['score_pct'] = df['score'].map(lambda x: f'{x:.1%}')

# applymap / map (DataFrame element-wise) — Pandas 2.0+ uses map
df.map(lambda x: round(x, 2) if isinstance(x, float) else x)

# vectorized string operations via .str accessor
df['name'].str.upper()
df['name'].str.len()
df['name'].str.replace('Alice','A.')
df['text'].str.split(' ').str[0]   # first word

# Binning / discretization
pd.cut(df['age'], bins=[0,18,35,60,100],
       labels=['child','adult','middle','senior'])
pd.qcut(df['score'], q=4, labels=['Q1','Q2','Q3','Q4'])  # quartile bins
```

■ *ML RELEVANCE: apply is used for feature engineering functions. pd.cut discretizes continuous features into categorical bins. .str operations process text features.*

■ 3.10 Sorting & Ranking

```
df.sort_values('score')                          # ascending
df.sort_values('score', ascending=False)         # descending
df.sort_values(['category','score'], ascending=[True,False])  # multi-col

df.sort_index()                       # sort by row index
df.sort_index(axis=1)                 # sort column names alphabetically

df['rank'] = df['score'].rank(ascending=False)   # rank (1=highest)
df['rank'] = df['score'].rank(method='dense')    # no gaps in ranks

df.nlargest(5, 'score')    # top 5 rows by score
df.nsmallest(5, 'age')     # bottom 5 rows by age
```

■ *ML RELEVANCE: Sorting is used to inspect top/bottom predictions, rank feature importances, and implement rank-based metrics.*

■ 3.11 String Operations (.str accessor)

```
s = df['name']
s.str.lower()            s.str.upper()
s.str.strip()            s.str.lstrip()  s.str.rstrip()
s.str.len()              # length of each string
s.str.contains('ali', case=False)   # bool mask
s.str.startswith('A')    s.str.endswith('e')
s.str.replace('old','new', regex=False)
s.str.split(',')         # split into list
s.str.split(',', expand=True)  # split into columns
s.str.get(0)             # get element 0 of each list
s.str[0:3]               # slice each string
s.str.extract(r'(\d+)')  # regex extract group
s.str.findall(r'\d+')   # find all matches
s.str.count('a')         # count occurrences
s.str.cat(sep=', ')      # concatenate all strings
```

■ *ML RELEVANCE: String operations preprocess NLP features: tokenization, casing normalization, regex extraction of numeric values from mixed-type fields.*

■ 3.12 DateTime Operations

```
# Parse strings to datetime
df['date'] = pd.to_datetime(df['date_str'])
df['date'] = pd.to_datetime(df['date_str'], format='%Y-%m-%d')

# DateTime component extraction (.dt accessor)
df['year']    = df['date'].dt.year
df['month']   = df['date'].dt.month
df['day']     = df['date'].dt.day
df['weekday'] = df['date'].dt.dayofweek  # 0=Mon, 6=Sun
df['hour']    = df['date'].dt.hour
df['quarter'] = df['date'].dt.quarter

# Time deltas
df['days_since'] = (pd.Timestamp.now() - df['date']).dt.days

# Resample time series
df.set_index('date').resample('M')['value'].mean()  # monthly mean

# Date ranges
pd.date_range('2024-01-01', periods=365, freq='D')  # daily for 1 year
```

■ *ML RELEVANCE: DateTime feature extraction (month, weekday, hour) creates cyclical temporal features. Resampling aggregates time-series data to uniform intervals for sequence models.*

SECTION 4: MATPLOTLIB

Matplotlib is the foundational plotting library. pyplot provides a state-machine interface; the OO API (fig, ax) is preferred for ML visualizations.

```
import matplotlib.pyplot as plt
import numpy as np
```

■ 4.1 Figure & Axes Setup

```
# OO API — recommended (explicit, composable)
fig, ax = plt.subplots()                    # single plot
fig, ax = plt.subplots(figsize=(10, 6))     # custom size (inches)

# Multiple subplots
fig, axes = plt.subplots(2, 3, figsize=(15, 8))  # 2 rows, 3 cols
ax = axes[0, 1]   # access subplot at row 0, col 1

# Display / save
plt.show()                                  # display in notebook/IDE
plt.tight_layout()                          # fix overlapping labels
fig.savefig('plot.png', dpi=150, bbox_inches='tight')
fig.savefig('plot.pdf')                     # vector format

# Close figure (free memory)
plt.close(fig)
plt.close('all')

# Style
plt.style.use('seaborn-v0_8-whitegrid')    # clean background
plt.rcParams['figure.dpi'] = 100           # global setting
```

■ *ML RELEVANCE: fig, ax = plt.subplots() is the standard pattern. Always call tight_layout() and savefig() to export training curves and evaluation plots.*

■ 4.2 Line & Scatter Plots

```
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Line plot
ax.plot(x, y)
ax.plot(x, y, color='blue', linewidth=2, linestyle='--',
        label='sin(x)', marker='o', markersize=3)

# Multiple lines
ax.plot(x, np.sin(x), label='sin')
ax.plot(x, np.cos(x), label='cos', linestyle=':')

# Scatter plot
ax.scatter(x, y)
ax.scatter(x, y, c=y, cmap='viridis',    # color-mapped
           s=50, alpha=0.6)               # size=50, transparency

# Annotations
ax.axhline(y=0, color='k', linewidth=0.5)    # horizontal line
ax.axvline(x=5, color='r', linestyle=':')    # vertical line
ax.fill_between(x, y-0.1, y+0.1, alpha=0.2) # shaded region

# Labels
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.set_title('Training Curve')
ax.legend(loc='best')
ax.grid(True, alpha=0.3)
ax.set_xlim(0, 10)
ax.set_ylim(-1.5, 1.5)
```

■ *ML RELEVANCE: Training/validation loss curves are the primary diagnostic tool in ML. Plot both on the same axes (two ax.plot calls) to detect overfitting.*

■ 4.3 Histograms, Bar Charts & Box Plots

```
# Histogram — distribution of values
data = np.random.randn(1000)
ax.hist(data, bins=30, edgecolor='black', color='steelblue', alpha=0.7)
ax.hist(data, bins=30, density=True)   # normalize to probability density

# Bar chart
categories = ['ClassA','ClassB','ClassC']
counts      = [300, 500, 200]
ax.bar(categories, counts, color=['#4f8ef7','#3ecf8e','#f7c948'])
ax.bar(categories, counts, width=0.5)  # bar width

# Horizontal bar
ax.barh(categories, counts)

# Box plot — quartiles + outliers
ax.boxplot([np.random.randn(100) for _ in range(4)],
           labels=['A','B','C','D'])

# Violin plot (distribution shape)
ax.violinplot([np.random.randn(100) for _ in range(4)])

# Error bars
means = [0.9, 0.8, 0.85]
stds  = [0.05, 0.03, 0.04]
ax.bar(categories, means, yerr=stds, capsize=5)
```

■ *ML RELEVANCE: Histograms reveal feature distributions and skewness before normalization. Bar charts display class counts (imbalance detection). Box plots compare metric distributions across folds.*

■ 4.4 Subplots & Layouts

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# Access subplots
axes[0,0].plot(x, np.sin(x), label='sin')
axes[0,0].set_title('Sine')

axes[0,1].hist(np.random.randn(500), bins=20)
axes[0,1].set_title('Gaussian')

axes[1,0].scatter(x, np.cos(x), c=x, cmap='plasma')
axes[1,0].set_title('Cosine Scatter')

axes[1,1].bar(['A','B','C'], [3,5,4])
axes[1,1].set_title('Bar')

plt.tight_layout()   # prevent overlap

# Flatten for iteration
for ax in axes.flat:
    ax.set_xlabel('x')

# Share axes
fig, axes = plt.subplots(1, 3, sharey=True, figsize=(12,4))

# GridSpec — irregular layouts
import matplotlib.gridspec as gridspec
gs = gridspec.GridSpec(2, 3)
ax_main = fig.add_subplot(gs[0, :])   # spans all 3 cols in row 0
ax_sub  = fig.add_subplot(gs[1, 0])   # row 1, col 0
```

■ *ML RELEVANCE: Subplot panels display multiple training metrics simultaneously: loss, accuracy, gradient norms, learning rate schedule — all on one figure.*

■ 4.5 Customization & Heatmaps

```
# Colormaps
# Sequential:   viridis, plasma, inferno, magma, Blues
# Diverging:    RdBu, seismic, coolwarm (good for error maps)
# Qualitative:  tab10, Set1, Set2 (categorical classes)

# Colorbar
sc = ax.scatter(x, y, c=z, cmap='viridis')
fig.colorbar(sc, ax=ax, label='Intensity')

# Heatmap (e.g., confusion matrix)
matrix = np.array([[50,2,1],[3,45,2],[1,3,48]])
im = ax.imshow(matrix, cmap='Blues')
fig.colorbar(im)
ax.set_xticks([0,1,2]); ax.set_xticklabels(['Cat','Dog','Bird'])
ax.set_yticks([0,1,2]); ax.set_yticklabels(['Cat','Dog','Bird'])

# Annotate each cell with value
for i in range(3):
    for j in range(3):
        ax.text(j, i, matrix[i,j], ha='center', va='center')

# Font sizes
ax.set_title('Confusion Matrix', fontsize=14, fontweight='bold')
ax.tick_params(labelsize=10)

# Legend customization
ax.legend(loc='upper right', fontsize=10, framealpha=0.8)
```

■ *ML RELEVANCE: Confusion matrix heatmaps are the standard for multi-class classifier evaluation. ax.imshow renders any 2D numpy array as a heatmap (correlation matrices, attention weights).*

■ 4.6 Saving Figures

```
# PNG — raster, good for reports and web
fig.savefig('results/training_curve.png', dpi=150, bbox_inches='tight')

# PDF/SVG — vector, scalable, good for papers
fig.savefig('results/figure1.pdf', bbox_inches='tight')
fig.savefig('results/figure1.svg')

# Ensure output directory exists
import os
os.makedirs('results', exist_ok=True)
fig.savefig('results/plot.png', dpi=150, bbox_inches='tight')
plt.close(fig)   # free memory — IMPORTANT in loops
```

■ *ML RELEVANCE: Always save figures with bbox_inches='tight' to prevent label clipping. Close figures in training loops to prevent memory leaks. PDF/SVG for journal submission.*

SECTION 5: SCIKIT-LEARN

scikit-learn is the standard ML library for classical algorithms. It defines a unified API: all estimators implement fit(), transform(), and/or predict().

■ 5.1 Preprocessing

```
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, RobustScaler,
    LabelEncoder, OneHotEncoder, OrdinalEncoder,
    PolynomialFeatures, PowerTransformer
)

# StandardScaler: z = (x - mean) / std  → mean=0, std=1
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # fit on train ONLY
X_test_s  = scaler.transform(X_test)         # apply same params to test
scaler.mean_      # per-feature means learned from train
scaler.scale_     # per-feature stds

# MinMaxScaler: z = (x - min) / (max - min)  → [0,1]
mm = MinMaxScaler(feature_range=(0,1))
mm.fit_transform(X_train)

# RobustScaler: uses median & IQR — outlier-resistant
RobustScaler().fit_transform(X_train)

# Label encoding — ordinal integers for target variable
le = LabelEncoder()
y_encoded = le.fit_transform(['cat','dog','bird','cat'])  # [1,2,0,1]
le.classes_    # ['bird','cat','dog']
le.inverse_transform([1,2])   # ['cat','dog']

# OneHotEncoder — categorical features
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
X_ohe = ohe.fit_transform(X_cat)   # returns dense array

# PolynomialFeatures — expand features with interactions
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)   # adds x1^2, x2^2, x1*x2 ...

# Power transform — make features more Gaussian
pt = PowerTransformer(method='yeo-johnson')  # handles negatives
pt.fit_transform(X)
```

■ *ML RELEVANCE: CRITICAL: Always fit scalers on X_train only, then transform X_test. Fitting on the full dataset causes data leakage — the test set contaminates the scaler parameters.*

■ 5.2 Train/Test Split

```
from sklearn.model_selection import train_test_split

# Basic split — 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Stratified split — preserves class proportions
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# Three-way split — train / validation / test
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.3, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(X_tmp, y_tmp, test_size=0.5)

print(X_train.shape, X_val.shape, X_test.shape)
```

■ *ML RELEVANCE: stratify=y is mandatory for imbalanced datasets — without it, a random split may place all minority-class samples in test. Always set random_state for reproducibility.*

■ 5.3 The Estimator API Pattern

```
# ALL scikit-learn models follow this interface:

# 1. Instantiate with hyperparameters
model = SomeModel(param1=value1, param2=value2)

# 2. Fit on training data
model.fit(X_train, y_train)

# 3. Predict
y_pred  = model.predict(X_test)         # class labels
y_proba = model.predict_proba(X_test)   # class probabilities (classifiers)
y_score = model.decision_function(X_test)  # raw scores (SVMs)

# 4. Evaluate (score method — accuracy for classifiers, R2 for regressors)
model.score(X_test, y_test)

# Learned parameters (available after fit)
model.coef_          # linear model weights
model.intercept_     # bias
model.feature_importances_  # tree-based models
model.n_iter_        # iterations to convergence

# Transformers additionally implement:
scaler.fit(X_train)         # learn parameters
scaler.transform(X_test)    # apply transformation
scaler.fit_transform(X)     # fit + transform in one call
scaler.inverse_transform(X_scaled)  # reverse transformation
```

■ *ML RELEVANCE: The consistent API means switching from LogisticRegression to RandomForest changes only line 1. This enables rapid algorithm comparison.*

■ 5.4 Key Algorithms

```
# ■■ CLASSIFICATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

LogisticRegression(C=1.0, max_iter=1000, multi_class='auto')
RandomForestClassifier(n_estimators=100, max_depth=None, random_state=42)
GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3)
SVC(kernel='rbf', C=1.0, gamma='scale', probability=True)
KNeighborsClassifier(n_neighbors=5, metric='euclidean')

# ■■ REGRESSION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR

LinearRegression(fit_intercept=True)
Ridge(alpha=1.0)          # L2 regularization
Lasso(alpha=0.1)          # L1 regularization (sparse solutions)
ElasticNet(alpha=0.1, l1_ratio=0.5)  # L1 + L2

# ■■ CLUSTERING ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering

KMeans(n_clusters=5, random_state=42, n_init=10)
DBSCAN(eps=0.5, min_samples=5)

# ■■ DIMENSIONALITY REDUCTION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

PCA(n_components=50)                  # linear, preserves variance
TSNE(n_components=2, random_state=42)  # nonlinear, for visualization
```

■ *ML RELEVANCE: Always baseline with LogisticRegression/LinearRegression before complex models. RandomForest is a strong default. Regularization (Ridge/Lasso) prevents overfitting.*

■ 5.5 Evaluation Metrics

```
from sklearn import metrics

# ■■ CLASSIFICATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
metrics.accuracy_score(y_true, y_pred)           # fraction correct
metrics.precision_score(y_true, y_pred, average='weighted')
metrics.recall_score(y_true, y_pred, average='weighted')
metrics.f1_score(y_true, y_pred, average='weighted')
metrics.balanced_accuracy_score(y_true, y_pred)  # for imbalanced data

# Classification report — all metrics at once
print(metrics.classification_report(y_true, y_pred))

# Confusion matrix
cm = metrics.confusion_matrix(y_true, y_pred)

# ROC-AUC (binary)
metrics.roc_auc_score(y_true, y_score)

# ROC curve (for plotting)
fpr, tpr, thresholds = metrics.roc_curve(y_true, y_score)

# ■■ REGRESSION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
metrics.mean_absolute_error(y_true, y_pred)      # MAE
metrics.mean_squared_error(y_true, y_pred)       # MSE
np.sqrt(metrics.mean_squared_error(y_true, y_pred))  # RMSE
metrics.r2_score(y_true, y_pred)                 # R² (1=perfect)
metrics.mean_absolute_percentage_error(y_true, y_pred)  # MAPE

# ■■ CLUSTERING ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
metrics.silhouette_score(X, labels)              # [-1,1] higher=better
metrics.adjusted_rand_score(y_true, y_pred)      # cluster agreement
```

■ *ML RELEVANCE: Accuracy is misleading on imbalanced data — use F1, balanced_accuracy, or ROC-AUC instead. classification_report is the standard first evaluation step.*

■ 5.6 Pipelines

```
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Pipeline — chain transformers + final estimator
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model',  LogisticRegression(max_iter=1000))
])

# Shorthand (auto-names steps)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))

# Pipeline behaves like any estimator
pipe.fit(X_train, y_train)
pipe.predict(X_test)
pipe.score(X_test, y_test)

# Access individual steps
pipe.named_steps['scaler'].mean_
pipe['model'].coef_

# ColumnTransformer — different preprocessing per column type
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

num_cols = ['age', 'score']
cat_cols = ['city', 'category']

ct = ColumnTransformer([
    ('num', StandardScaler(),      num_cols),
    ('cat', OneHotEncoder(),       cat_cols),
], remainder='passthrough')    # pass other cols through unchanged

full_pipe = Pipeline([
    ('preprocessing', ct),
    ('model', RandomForestClassifier())
])
```

■ *ML RELEVANCE: Pipelines prevent data leakage (scaler is fit only on train fold during CV), enable one-call train/predict, and simplify deployment. ColumnTransformer is standard for mixed numeric/categorical data.*

■ 5.7 Cross-Validation & Hyperparameter Search

```
from sklearn.model_selection import (
    cross_val_score, KFold, StratifiedKFold,
    GridSearchCV, RandomizedSearchCV
)

# k-fold cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring='f1_weighted')
print(f'CV F1: {scores.mean():.3f} ± {scores.std():.3f}')

# Scoring strings:
# Classification: 'accuracy','f1','precision','recall','roc_auc'
# Regression:     'r2','neg_mean_squared_error','neg_mae'

# Grid Search — exhaustive (small grids)
param_grid = {
    'model__C':           [0.01, 0.1, 1, 10],
    'model__penalty':     ['l1','l2'],
    'scaler__with_std':   [True, False]
}
gs = GridSearchCV(pipe, param_grid, cv=5, scoring='f1_weighted',
                  n_jobs=-1, verbose=1)
gs.fit(X_train, y_train)
gs.best_params_    # best hyperparameter combination
gs.best_score_     # best CV score
gs.best_estimator_ # fitted model with best params

# Randomized Search — efficient (large grids)
from scipy.stats import loguniform
param_dist = {
    'model__C': loguniform(0.001, 100),
    'model__max_iter': [100, 500, 1000]
}
rs = RandomizedSearchCV(pipe, param_dist, n_iter=50,
                        cv=5, scoring='roc_auc', random_state=42)
rs.fit(X_train, y_train)
```

■ *ML RELEVANCE: Cross-validation gives an unbiased generalization estimate. Always use StratifiedKFold for classification. GridSearchCV with n_jobs=-1 parallelizes across all CPU cores.*

QUICK IMPORT REFERENCE

```
# ■■ Standard library ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import os, sys, json, csv, math, random, time
from collections import defaultdict, Counter, namedtuple, deque
from functools import reduce, partial, lru_cache
from itertools import chain, product, combinations, islice
from pathlib import Path

# ■■ Core numerical stack ■■■■■■■■■■■■■■■■■■■■■■■■■■
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.colors import Normalize

# ■■ scikit-learn ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.model_selection import (
    train_test_split, cross_val_score,
    StratifiedKFold, GridSearchCV, RandomizedSearchCV
)
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, RobustScaler,
    LabelEncoder, OneHotEncoder, PolynomialFeatures
)
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.compose import ColumnTransformer
from sklearn import metrics

# ■■ ML algorithms ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.linear_model import (
    LogisticRegression, LinearRegression, Ridge, Lasso, ElasticNet
)
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.ensemble import (
    RandomForestClassifier, RandomForestRegressor,
    GradientBoostingClassifier, GradientBoostingRegressor
)
from sklearn.svm import SVC, SVR
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# ■■ Deep learning (when ready) ■■■■■■■■■■■■■■■■■■■■
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
```

END OF DOCUMENT — PYTHON PRE-ML COMPREHENSIVE REFERENCE